

Relatório do trabalho final

Simulação de uma rede em anel com passagem de token sobre UDP

Professora: Prof. Cristina Moreira Nunes

Alunos: Lorenzo Schavinski Eulalio de Souza, Gabriel Guggiana Escouto, Joao Vitor da Silva Freitas

Data: 22/06/2026

1. Introdução

Em uma rede lógica em anel, as máquinas são organizadas em uma sequência circular. Cada participante conhece sua próxima máquina e encaminha para ela os pacotes recebidos. Quando o último elemento da sequência transmite, o pacote retorna ao primeiro, fechando o anel.

Neste trabalho, essa topologia é simulada sobre uma rede IP convencional. A comunicação utiliza UDP, mas os pacotes não são enviados livremente para qualquer participante. Depois que o anel é formado, cada máquina encaminha tokens e dados somente para sua sucessora lógica.

Para um anel formado por quatro máquinas, a ordem pode ser representada da seguinte forma:

A -> B -> C -> D -> A

O controle de acesso é realizado por um token, representado pelo pacote 1000. Em condições normais existe apenas um token circulando. Uma máquina precisa estar de posse dele para iniciar uma transmissão, evitando que várias mensagens sejam enviadas simultaneamente no anel.

Ao receber o token, a máquina consulta sua fila. Se não houver mensagens, aguarda o tempo configurado e o encaminha. Caso exista uma mensagem pendente, transmite apenas o primeiro item da fila e mantém o token retido até que os dados completem uma volta no anel.

Além da transmissão normal, a aplicação trata erros de mensagem, retransmissões, perda e duplicação de token, entrada de novos participantes e ausência de máquinas. Todas as ações relevantes são apresentadas no terminal para permitir o acompanhamento durante a execução.

2. Descrição geral da solução

Cada execução do programa representa uma máquina independente. O arquivo de configuração informa:

```
<apelido_da_máquina>  
<tempo_token_e_dados>  
<probabilidade_de_erro>  
<timeout_do_token>  
<tempo_mínimo_entre_tokens>
```

O apelido identifica a máquina e determina sua posição no anel. Os valores numéricos controlam o tempo de retenção do token, a probabilidade de corrupção de mensagens, o tempo usado para detectar a perda do token e o intervalo mínimo utilizado na detecção de duplicidade.

Na inicialização, cada máquina envia DISCOVER em broadcast na porta UDP 6000. Ao receber essa mensagem, as outras aplicações registram a máquina e respondem com HELLO, também em broadcast. Com os apelidos recebidos, cada participante monta uma lista em ordem alfabética.

Considerando a lista:

```
["A", "B", "C"]
```

as ligações são:

```
A envia para B  
B envia para C  
C envia para A
```

A primeira máquina da ordem é responsável por gerar o token inicial. A geração ocorre após uma janela de descoberta, evitando que várias aplicações iniciadas em momentos próximos se considerem sozinhas e criem tokens diferentes.

Depois da formação do anel, o funcionamento normal segue este ciclo:

1. Uma máquina recebe o token.
2. Se sua fila estiver vazia, ela encaminha o token.
3. Se houver mensagem, envia a primeira ao seu sucessor.
4. O pacote passa pelas demais máquinas.
5. O destino verifica o CRC e marca ACK ou NAK.
6. O pacote continua circulando até retornar à origem.
7. A origem trata o resultado e libera o token.

O programa também possui comandos para adicionar mensagens, visualizar a fila e o estado do anel, inserir um token adicional e retirar um token da rede.

3. Formação e atualização do anel

3.1 Descoberta inicial

Os pacotes de descoberta seguem exatamente os formatos:

```
10:<apelido>:<endereço IP>  
20:<apelido>:<endereço IP>
```

O código 10 representa DISCOVER, enquanto 20 representa HELLO.

Como UDP não garante que um datagrama seja entregue, a aplicação repete o DISCOVER durante a janela inicial. Essa repetição utiliza o mesmo pacote oficial e não altera o protocolo. O recebimento repetido é tratado de forma idempotente: a presença é atualizada, mas a máquina não é adicionada duas vezes.

Depois que os participantes são conhecidos, todos utilizam a mesma regra de ordenação. Assim, máquinas que conhecem o mesmo conjunto calculam a mesma topologia e a mesma sucessora.

3.2 Entrada de novas máquinas

Uma máquina pode entrar depois que o anel já está funcionando. Ela envia DISCOVER, recebe as respostas HELLO e é inserida na posição correspondente ao seu apelido.

Se a nova máquina possuir o menor apelido, ela passa a ser a controladora. Entretanto, não gera outro token caso já tenha observado token ou dados circulando.

Conforme solicitado na especificação, a entrada deve ocorrer em um momento em que não existam dados circulando, ou seja, quando somente o token estiver passando pelo anel.

3.3 Detecção de máquinas ausentes

A especificação não define um pacote para informar a saída de uma máquina. Para preservar a interoperabilidade, não foi criado um tipo adicional de datagrama.

A presença é verificada reutilizando DISCOVER e HELLO. As máquinas enviam anúncios periodicamente e guardam o instante da última confirmação de cada participante. Se uma máquina permanecer sem responder por determinado período, ela é removida da lista e o anel é recalculado.

Também existe uma indicação complementar: se um pacote unicast completa a volta mantendo o controle maquina inexistente, significa que nenhum destino o reconheceu.

Nesse caso, a origem informa a ausência e pode remover esse participante de sua topologia.

Como cada aplicação mantém sua própria lista, pode existir um pequeno intervalo de atualização após uma entrada ou saída. Entretanto, todas convergem porque utilizam os mesmos anúncios, o mesmo critério de presença e a mesma ordenação alfabética.

4. Funcionamento do token e transmissão de dados

4.1 Circulação do token

O token possui o formato:

1000

Ele não contém campos adicionais. Ao recebê-lo, a máquina informa sua chegada e verifica a fila de mensagens.

Se a fila estiver vazia, o token é mantido pelo tempo definido na configuração e depois encaminhado. Se uma mensagem for adicionada durante esse intervalo, a fila é consultada novamente e a transmissão pode ser iniciada antes do repasse.

Se a máquina estiver sozinha, ela mantém o token em vez de enviá-lo continuamente para si mesma. Quando outra máquina entra, o sucessor é recalculado e a circulação é retomada.

O valor de “tempo_token_e_dados” é utilizado para definir o tempo de retenção do token quando a fila está vazia. Durante a circulação dos dados, optamos por não aplicar um atraso artificial em cada repasse. A localização do pacote pode ser acompanhada pelos registros de envio, recepção e encaminhamento exibidos no terminal, enquanto o token permanece retido pela origem até o retorno dos dados.

4.2 Pacote de dados

O formato definido para dados é:

2000:<origem>:<destino>:<controle>:<CRC>:<mensagem>

O pacote é criado inicialmente com o controle maquinainexistente. A mensagem pode conter o caractere “:”, pois o parser limita a separação do cabeçalho e preserva todo o restante como conteúdo.

As máquinas intermediárias não modificam o pacote. Elas encaminham os mesmos bytes recebidos para sua sucessora. Somente o destino altera o campo de controle.

4.3 Comunicação unicast

No unicast, o campo de destino contém o apelido de uma única máquina. Quando o pacote chega ao destino:

1. O CRC da mensagem recebida é recalculado.
2. A origem e a mensagem são exibidas.
3. O controle é alterado para ACK se o CRC estiver correto.
4. O controle é alterado para NAK se houver divergência.
5. O pacote é encaminhado e continua sua volta.

Quando retorna à origem:

- ACK confirma a entrega e remove a mensagem da fila;
- maquinainexistente informa que o destino não respondeu e remove a mensagem;
- NAK mantém a mensagem na fila para uma retransmissão.

Depois do tratamento, o token é encaminhado para a próxima máquina.

4.4 Comunicação broadcast

No broadcast, o campo de destino recebe o valor:

BROADCAST

Cada máquina, exceto a origem, exibe a mensagem e encaminha o pacote sem alterar o campo de controle. Quando ele retorna à origem, o broadcast é considerado concluído, a mensagem é removida da fila e o token é liberado.

O módulo de falhas não corrompe mensagens broadcast e o controle permanece maquinainexistente, conforme definido.

5. CRC, inserção de falhas e retransmissão

5.1 Cálculo do CRC

O controle de erro utiliza CRC32. A origem calcula o valor sobre os bytes originais da mensagem e o envia como número decimal no cabeçalho.

O cálculo utilizado é equivalente a:

`CRC = zlib.crc32(mensagem) & 0xFFFFFFFF`

O CRC cobre somente os bytes da mensagem. Os demais campos do pacote não participam do cálculo.

5.2 Inserção probabilística de erros

Depois de calcular o CRC, a origem consulta a probabilidade configurada. Quando uma falha é sorteada, um byte aleatório da mensagem é alterado antes do envio, mas o pacote mantém o CRC calculado sobre o conteúdo original.

Ao receber os dados, o destino calcula novamente o CRC. Como os bytes foram modificados, o resultado não coincide com o campo recebido e o pacote é marcado com NAK.

Não é inserida falha quando:

- a probabilidade configurada é zero;
- a mensagem está vazia;
- a transmissão é um broadcast;
- a mensagem está sendo retransmitida.

5.3 Retransmissão

Quando a origem recebe um primeiro NAK, ela:

- informa que haverá retransmissão;
- mantém a mensagem na cabeça da fila;
- encaminha o token;
- aguarda a próxima passagem do token.

Na próxima oportunidade, envia novamente a mensagem original, sem corrupção, e reinicia o controle como maquina inexistente.

A retransmissão é realizada somente uma vez. Se um novo NAK ocorrer, a mensagem é descartada e o token é encaminhado.

5.4 Proteção contra pacote sem retorno

Ao iniciar uma transmissão, a origem cria um temporizador usando o valor de `timeout_token` do arquivo de configuração. Se o pacote nunca retornar, a máquina abandona a tentativa e libera o token, evitando que todo o anel fique parado.

Cada tentativa recebe um identificador local. Dessa forma, um timer ou retorno antigo não pode remover uma mensagem posterior nem produzir um segundo token.

6. Estruturas, classes, threads e sincronização

A solução foi dividida em classes e módulos com responsabilidades diferentes, mas o estado principal de cada máquina é mantido por um único objeto Node.

6.1 Estruturas de dados

As principais estruturas são:

- um dicionário de membros, associando apelidos a endereços;
- uma lista ordenada logicamente para calcular sucessor e controladora;
- uma fila FIFO com capacidade máxima de dez mensagens;
- um registro do último anúncio recebido de cada participante;
- estados que indicam posse do token, transmissão ativa e pedidos pendentes.

Cada item da fila armazena o destino, a mensagem original e informações necessárias para controlar a retransmissão.

6.2 Principais classes

As classes centrais são:

- Node: representa uma máquina e coordena o protocolo;
- Ring: mantém os membros e calcula a ordem e o sucessor;
- MessageQueue: implementa a fila de mensagens;
- Transport: encapsula o socket UDP;
- TokenMonitor: acompanha o timeout do token;
- Console: recebe os comandos do usuário.

Os formatos de pacotes, o CRC e a inserção de falhas foram separados em módulos próprios para evitar que diferentes partes do programa montassem datagramas de maneiras incompatíveis.

6.3 Threads

A aplicação utiliza:

- a thread principal, responsável pelo console;
- uma thread receptora, bloqueada aguardando datagramas UDP;
- uma thread de motor, responsável pelas alterações do estado;
- uma thread de monitoramento do token;
- temporizadores para ações que precisam ocorrer posteriormente.

As threads auxiliares não alteram livremente o estado principal. Elas publicam eventos em uma "queue.Queue" e a thread do motor trata um evento por vez.

Esse modelo serializa situações que poderiam ocorrer simultaneamente, como:

- chegada de um ACK e disparo de timeout;
- comando de remoção enquanto dados circulam;
- token recebido enquanto existe um timer de repasse;
- mudança da topologia durante o monitoramento.

O pequeno estado interno do monitor é protegido por "threading.Lock", pois é acessado pela recepção, pelo motor e pela própria thread de monitoramento. A saída no terminal também utiliza um lock para impedir que mensagens de threads diferentes se misturem.

7. Controle de falhas e do token

7.1 Máquina controladora

A máquina de menor apelido na topologia atual é considerada a controladora. Ela é responsável por verificar se o token continua circulando corretamente.

Não existe uma mensagem específica de eleição. Quando a lista muda, todas as máquinas ordenam os apelidos novamente e chegam à mesma controladora.

7.2 Token perdido

O monitor acompanha o tempo desde a última atividade de token ou dados. Os pacotes de dados também contam como atividade porque, enquanto eles circulam, o token está retido logicamente pela origem.

Se o tempo configurado for ultrapassado, a máquina confirma se:

- ainda é a controladora;
- não possui o token;
- não está aguardando o retorno de dados;
- existem outros participantes no anel.

Se essas condições forem satisfeitas, informa TOKEN PERDIDO, incrementa o contador e gera um novo token.

7.3 Retirada do token

O comando removetoken permite demonstrar a perda:

- se a máquina possui o token, ele é retirado;
- se dados estão circulando, a retirada ocorre após a conclusão;

- se a máquina não possui o token, o pedido fica pendente e o próximo token recebido é removido.

7.4 Token duplicado

A controladora identifica duplicidade quando:

- já está segurando um token e recebe outro;
- recebe tokens em intervalo menor que o tempo mínimo configurado.

O token excedente é consumido, o contador de duplicidade é atualizado e o token legítimo continua circulando. Também é possível inserir um token adicional manualmente com o comando gentoken

7.5 Visualização

O terminal informa os eventos importantes:

[A] recebeu o token
[A] enviando token para B
[C] DADOS de A: "Teste erro" -> NAK
[A] RETRANSMISSAO: NAK recebido de C
[A] TOKEN PERDIDO detectado (timeout) -> gerando novo token
[A] TOKEN DUPLICADO detectado -> removido da rede

O comando status mostra a ordem do anel, o sucessor, a controladora, a posse do token, o tamanho da fila e os contadores de perda e duplicidade.

8. Exemplo simplificado de execução real

O trecho a seguir resume uma execução com três máquinas organizadas no anel

A -> B -> C -> A.

8.1 Formação do anel e circulação do token

[A] DISCOVER de B, respondendo HELLO
[A] DISCOVER de C, respondendo HELLO
[A] gerou o token inicial
[A] recebeu o token
[A] enviando token para B
[B] recebeu o token
[B] enviando token para C

8.2 Envio com confirmação

[B] enviando DADOS para A: "Ola A"
[C] repassando DADOS de B para A
[A] DADOS de B: "Ola A" -> ACK
[B] mensagem entregue com sucesso
[B] enviando token para C

8.3 Erro e retransmissão

[A] enviando DADOS para C com erro: "Teste erro"
[B] repassando DADOS de A para C
[C] CRC não confere -> NAK
[A] NAK recebido: mensagem será retransmitida

[A] recebeu novamente o token
[A] retransmitindo DADOS para C sem erro
[C] DADOS de A: "Teste erro" -> ACK
[A] mensagem entregue com sucesso

8.4 Recuperação de token perdido e duplicado

[B] token retirado da rede
[A] TOKEN PERDIDO detectado -> gerando novo token
[A] novo token inserido na rede

[C] token adicional inserido
[A] TOKEN DUPLICADO detectado -> removido da rede

9. Conclusão

O trabalho implementou uma rede lógica em anel sobre UDP, utilizando a passagem de token para coordenar o envio de mensagens. As máquinas são descobertas por broadcast, ordenadas alfabeticamente e conectadas de maneira circular.

Cada máquina possui uma fila de até dez mensagens e transmite somente a primeira quando recebe o token. Os dados percorrem o anel e retornam à origem antes da liberação do token, garantindo que apenas uma transmissão seja iniciada por vez.

A aplicação oferece unicast e broadcast, calcula CRC32, insere falhas probabilísticas, responde com ACK ou NAK e realiza uma única retransmissão. Também detecta destinos que não responderam e evita que uma tentativa sem retorno retenha o token indefinidamente.

A controladora monitora a atividade da rede, recupera tokens perdidos e elimina tokens duplicados. A topologia pode ser atualizada durante a execução pela entrada de novas

máquinas ou pela ausência de participantes, utilizando somente os pacotes DISCOVER e HELLO.

A organização com uma thread central de eventos reduziu problemas de concorrência entre recepção, comandos, temporizadores e monitoramento. As classes separam as responsabilidades de transporte, topologia, fila, controle de token e interface.

Por fim, os datagramas seguem os formatos definidos na especificação, sem novos códigos ou campos. Dessa forma, a aplicação mantém a compatibilidade necessária para se comunicar com implementações desenvolvidas por outros grupos.